

## Basic part 2 (venv setup) — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the proper terminal workflow), **Advanced** (how a real project automates it so nobody has to remember these steps by hand). Read Basic first even if you already know venvs — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and the exact pieces

---

### Basic Version

This exercise is about typing commands, not writing code. You need to do four things in order: make a new, empty Python space just for this project, turn it on, install one package into it, and write down what you installed.

Here are the four commands, in order:

1. `python -m venv .venv` — makes the new space.
2. `source .venv/bin/activate` — turns it on (Windows: `.venv\Scripts\activate` ).
3. `pip install requests` — installs one package.
4. `pip freeze > requirements.txt` — writes down what's installed.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

The mental model: `python -m venv .venv` creates an isolated Python environment (its own `site-packages` folder, separate from your system Python). Activating it ( `source .venv/bin/activate` ) changes which `python` and `pip` your shell resolves to, for the rest of that terminal session. From that point, anything you `pip install` only affects this project, not your whole system or any other project.

`pip freeze > requirements.txt` writes every installed package and its exact version to a file — this is what lets anyone (including future you) recreate the identical environment later with `pip install -r requirements.txt`.

Same four commands, with what to check after each one:

1. `python -m venv .venv` — check: a new `.venv/` folder now exists.
2. `source .venv/bin/activate` — check: your terminal prompt now shows `(.venv)` at the start.
3. `pip install requests` — check: `pip list` shows `requests` in the installed packages.
4. `pip freeze > requirements.txt` — check: opening `requirements.txt` shows a line like `requests==2.32.3`.

If step 2 doesn't seem to do anything visible, that's the most common place this goes wrong — see the [Solution](#) for what to check.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

Think about what a real project needs beyond "I typed 4 commands once, on my machine." Two problems show up fast: a teammate (or a CI server) has to type the exact same 4 commands correctly, in order, every single time — one typo or a skipped step and their environment silently doesn't match yours. And `pip freeze` pins what you happened to have installed *today*, not necessarily what you'll get if you run the same commands again in six months, once newer versions of `requests` exist.

That's the real design question underneath "how do I make a venv": **how do you make the setup process itself repeatable and automatic**, so it doesn't depend on a human typing 4 commands correctly, and so anyone on any machine ends up with the exact same environment. That's what a `Makefile` /script, or a modern tool like `uv`, is actually solving.

The pieces for automating the same 4 commands, so they run correctly with one command instead of four typed by hand:

- A `Makefile` target — a named recipe ( `make setup` ) that runs a list of shell commands in order, every time, identically.
- A shell script ( `setup.sh` ) with `set -e` at the top — this makes the script stop immediately if any single command in it fails, instead of plowing ahead with a broken environment.
- A lockfile-based tool such as `uv` — instead of you running `pip freeze` yourself, the tool writes an exact, reproducible lockfile ( `uv.lock` ) automatically every time you add a package.

Sketch a `Makefile` with one target, `setup`, that runs your 4 commands, before checking Hint 2.

**Difference between Basic, Intermediate, and Advanced:** Basic names the 4 commands you need to run. Intermediate explains what each command actually does under the hood and how to verify it worked. Advanced asks the harder question — not "how do I set up a venv" but "how do I make this setup process repeatable for other people and other machines, without relying on someone typing it correctly" — which is the difference between a workflow that works once for you and one a real team can depend on.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

```
open a terminal, go to your project folder
create a venv named .venv
turn it on
install "requests" with pip
save the list of installed packages to requirements.txt
open requirements.txt and look for "requests" in it
```

Here's almost the whole thing — just try running it and reading it line by line:

```
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
```

**Expected output:** none of these 4 commands print much on their own — `python -m venv .venv` and `source .venv/bin/activate` are silent when they succeed, and `pip install requests` prints a few lines ending in something like `Successfully installed requests-2.32.3 ...`. Run `cat requirements.txt` afterward to actually see a result — see the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

```
in your project folder:
python -m venv .venv
source .venv/bin/activate      (or .venv\Scripts\activate on Windows)
pip install requests
pip freeze > requirements.txt

confirm:
prompt shows (.venv)
requirements.txt contains a line starting with "requests=="
```

The commands, plus how to check each result:

```
python -m venv .venv
ls .venv                      # should show bin/, lib/, etc.

source .venv/bin/activate
which python                   # should now point inside .venv/

pip install requests
pip show requests              # confirms it installed correctly

pip freeze > requirements.txt
cat requirements.txt           # should list requests==<version>
```

**Expected output** of the last line, `cat requirements.txt` :

```
certifi==2024.8.30
charset-normalizer==3.3.2
idna==3.8
requests==2.32.3
urllib3==2.2.3
```

(Exact version numbers will differ depending on when you run this — `pip freeze` also lists `requests`'s own dependencies, not just `requests` itself, which is why you see 5 lines instead of 1.)

If activation seems to silently do nothing, see the [Solution](#) for the most common cause.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

```
make a file called Makefile with a target "setup" that:
    runs python -m venv .venv
    runs pip install (using the venv's own pip directly, no activation needed)
    runs pip freeze > requirements.txt
    prints a friendly "done" message

now the whole exercise becomes:
    one teammate types: make setup
    same 4 things happen, in the same order, every time
    no one has to remember the exact command names or the right order
```

A start on the `Makefile` version — deliberately incomplete, fill in the rest yourself:

```
.PHONY: setup
setup:
    python -m venv .venv
    .venv/bin/pip install requests
    # what's the 4th command? add it here, writing to requirements.txt
```

Note it uses `.venv/bin/pip` directly instead of `source .venv/bin/activate` first — a `Makefile` runs each line in its own fresh shell, so an `activate` in one line wouldn't carry over to the next line anyway. Calling the venv's own `pip` binary by its full path sidesteps that problem entirely.

Finish the target, run `make setup`, and compare against all 3 approaches in the [Solution](#).

**Difference between Basic, Intermediate, and Advanced:** same 4 underlying actions at 3 completeness levels, shown here both as pseudocode and as near-complete code — Basic proves you can do them by hand, Intermediate adds a way to confirm each step actually worked, Advanced collapses the whole thing into one command someone else can run without knowing the 4 steps exist, and without a human typing anything at all.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)